

Redis Event Loop

Redis uses a **single-threaded event-driven architecture** (for command execution), conceptually similar to Node.js.

Instead of creating one thread per client connection, Redis uses:

```
None
Single main thread
+
Event Loop
+
epoll() / I/O multiplexing
```

This is one of the key reasons Redis achieves extremely high throughput and low latency.

1. Core Idea

Redis handles many TCP clients concurrently by:

```
None
Accept connections asynchronously
Wait for socket readiness events
Process ready sockets quickly
Return to event loop
```

One thread can manage thousands of connections.

2. Why Redis Uses This Model

Avoids problems of thread-per-connection systems:

- Context switching overhead
- Lock contention
- High memory usage
- Complex synchronization

None

Redis prefers simple + fast + predictable execution

3. Main Responsibilities of Redis Event Loop

✓ Accept New Client Connections

When new TCP clients connect:

None

listen socket becomes readable

Redis accepts new sockets.

✓ Process Existing Client Commands

For connected clients:

None

GET key

SET key value

INCR counter

LPUSH queue item

Redis reads commands, executes them, writes responses.

4. High-Level Event Loop Flow

Step 1: Initialize Socket Types

Redis prepares networking abstractions.

Step 2: Create Event Loop

Internal event loop object:

None

server.el

This manages registered file descriptor events.

Step 3: Bind Address and Port

Redis starts listening on configured IP/port.

Example:

None

```
0.0.0.0:6379
```

Step 4: Register Accept Handlers

Listen socket gets callback/handler for new clients.

Step 5: Wait for Ready Events

Using Linux:

None

```
epoll_wait()
```

Kernel returns ready sockets.

Step 6: Process Returned Events

Redis loops through ready events:

None

Readable socket → read command

Writable socket → send response

Listen socket → accept client

5. Simplified Pseudocode

None

```
while server_running:

    ready_events = epoll_wait()

    for event in ready_events:

        if new_connection:

            accept_client()

        if client_readable:

            read_command()

            execute_command()

        if client_writable:

            send_response()
```

6. Why This Is Fast

✓ No Busy Waiting

Thread sleeps until OS reports work.

✓ No Lock Overhead

Single command execution thread means simpler concurrency.

✓ CPU Cache Friendly

Sequential event processing often improves locality.

✓ Thousands of Connections

Many idle clients can remain connected cheaply.

7. Important Internal Source Files (Conceptual)

Redis core networking/event logic commonly involves:

None

`server.c` → Main server logic
`ae.c / ae.h` → Event loop abstraction
`ae_epoll.c` → epoll backend
`connection.c` → Connection management
`socket.c / anet.c` → Networking helpers

(Names may vary by version.)

8. Redis vs Traditional Server

Model	Threads	Scalability	Complexity
Thread per connection	Many	Lower	High
Redis Event Loop	Few / Main loop	Very High	Efficient

9. Modern Note

Newer Redis versions also use extra threads for some networking/background tasks, but core command processing remains strongly event-loop driven.

None

Main command path still optimized for simplicity and speed

10. Interview Insight

If asked why Redis is fast:

None

```
Because Redis keeps data in RAM and uses a highly efficient
event loop with epoll instead of expensive
thread-per-connection processing.
```

Mental Model

None

```
OS tells Redis which sockets are ready.
```

```
Redis processes only those sockets quickly.
```

```
Repeat.
```

One-Line Summary

Redis uses a single-threaded event loop with epoll-based I/O multiplexing to efficiently accept connections and process commands for thousands of clients with minimal overhead.